# ASSIGNMENT-21.3

**NAME: G. MANI SHARAN**
**ROLL.NO: 2303A52148**
**BATCH: 41**

TASK 1 – AI-Assisted Expense Tracker

Initial prompt:

Create a simple Python expense tracker to store daily expenses with category and amount.

Initial code:

```
expenses = []

while True:
    name = input("Enter expense name: ")
    amount = float(input("Enter amount: "))
    category = input("Enter category: ")

    expenses.append((name, amount, category))

    choice = input("Add more? (y/n): ")
    if choice == 'n':
        break

print("Expenses:", expenses)
```

Output:

```
Enter expense name: food
Enter amount: 100
Enter category: grocieries
Add more? (y/n): y
Enter expense name: transport
Enter amount: 50
Enter category: travel
Add more? (y/n): n
Expenses: [('food', 100.0, 'grocieries'), ('transport', 50.0, 'travel')]
```

Improvement prompt:

Improve the expense tracker with monthly summary, input validation, and functions.

Improved code

```python
def add_expense(expenses):
    name = input("Enter expense name: ")

    while True:
        try:
            amount = float(input("Enter amount: "))
            break
        except:
            print("Invalid amount")

    category = input("Enter category: ")
    expenses.append({"name": name, "amount": amount, "category": category})


def show_summary(expenses):
    total = sum(e["amount"] for e in expenses)
    print("Total Monthly Expense:", total)


expenses = []

while True:
    add_expense(expenses)
    choice = input("Add more? (y/n): ")
    if choice == 'n':
        break

show_summary(expenses)
```

Output:


## Justification

- Functions → modular design
- Validation → avoids runtime errors

- Summary → real-world usability

TASK 2 – Git Version Control (Student Grade Calculator)

## Initialize Repo,Initial Code,Create Branch

```
> marks = list(map(int, input("Enter marks: ").split()))
>
> total = sum(marks)
> avg = total / len(marks)
> percentage = (total / (len(marks)*100)) * 100
>
> if avg >= 90:
>     grade = 'A'
> elif avg >= 75:
>     grade = 'B'
> elif avg >= 50:
>     grade = 'C'
> else:
>     grade = 'F'
>
> print("Average:", avg)
> print("Percentage:", percentage)
> print("Grade:", grade)
> EOF
Switched to a new branch 'feature-ai-grade'
```

## Ai prompt

Add grade classification and percentage calculation to the program.
Updated

```
marks = list(map(int, input("Enter marks: ").split()))

total = sum(marks)
avg = total / len(marks)
percentage = (total / (len(marks)*100)) * 100

if avg >= 90:
   grade = 'A'
elif avg >= 75:
   grade = 'B'
elif avg >= 50:
   grade = 'C'
else:
   grade = 'F'

print("Average:", avg)
print("Percentage:", percentage)
```

```
print("Grade:", grade)
```

Commit + Merge

```
--- grade.py ---
marks = list(map(int, input("Enter marks: ").split()))

total = sum(marks)
avg = total / len(marks)
percentage = (total / (len(marks)*100)) * 100

if avg >= 90:
    grade = 'A'
elif avg >= 75:
    grade = 'B'
elif avg >= 50:
    grade = 'C'
else:
    grade = 'F'

print("Average:", avg)
print("Percentage:", percentage)
print("Grade:", grade)
```

## Justification

- Branching → safe feature development
- Merge → integration workflow
- AI → faster feature addition

# TASK 3 – AI Commit Messages (Password Validation)

Setup code

```
import re



password = input("Enter password: ")



if len(password) < 8:

   print("Weak password")

elif not re.search(r'[!@#$%^&*]', password):

   print("Add special character")
```

```
else:

   print("Strong password")
```

Prompt

Generate commit message for adding password validation with length and special character rules.

AI Commit Message

Human Version

Added strong password validation rules

## Justification

- AI → more structured (uses “feat:” convention)
- Human → less descriptive

TASK 4 – Quiz Management System (Pair Programming)

## Student A Code

```
questions = {
   "Capital of India?": "Delhi",
   "2+2?": "4"
}

for q in questions:
   ans = input(q + " ")
   if ans == questions[q]:
       print("Correct")
   else:
       print("Wrong")
```

AI Prompt (Student B)

Add score calculation and input validation to quiz program.
Final code

```
questions = {
   "Capital of India?": "Delhi",
   "2+2?": "4"
}

score = 0

for q in questions:
   ans = input(q + " ").strip()

   if ans.lower() == questions[q].lower():
       print("Correct")
       score += 1
   else:
       print("Wrong")

print("Final Score:", score)
```

## Git Workflow

```
Mani Sharan-750XGK:~/Downloads/quiz-management$ git checkout -b feature-score
Switched to a new branch 'feature-score'

Mani Sharan-750XGK:~/Downloads/quiz-management$ git add .

Mani Sharan-750XGK:~/Downloads/quiz-management$ git commit -m "Added score calculation"
[feature-score 8ab91cd] Added score calculation
 1 file changed, 12 insertions(+), 1 deletion(-)

Mani Sharan-750XGK:~/Downloads/quiz-management$ git checkout main
Switched to branch 'main'

Mani Sharan-750XGK:~/Downloads/quiz-management$ git merge feature-score
Updating 2f4a7d1..8ab91cd
Fast-forward
 quiz.py | 12 ++++++++++-
 1 file changed, 11 insertions(+), 1 deletion(-)

Mani Sharan-750XGK:~/Downloads/quiz-management$ python quiz.py
Capital of India? Delhi
Correct
2+2? 5
Wrong
Final Score: 1

Mani Sharan-750XGK:~/Downloads/quiz-management$ git log --oneline --decorate --graph --all --max-
count=5
* 8ab91cd (HEAD -> main, feature-score) Added score calculation
* 2f4a7d1 Initial quiz management system
```

## Output:


## Justification

- Score → measurable output
- Validation → better UX
- Collaboration → real Git workflow